

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	Assessment:	Industry Problem solving task
Weightage:	30%	Total Marks:	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	K.MAHESH BABU	Reg. No.:	192525122
Section / Batch:	CSA-1214/2025	Date:	18/08/25

Instructions to Students

- Identify the relevant concepts of cache hierarchy, SRAM, DRAM, MRAM, NAND Flash, RRAM, memory latency, bandwidth, and virtual memory paging.
- Analyze the memory-access requirements of smartphones, cloud servers, autonomous vehicles, edge AI devices, and gaming consoles, considering latency, bandwidth, capacity, and power consumption.
- Apply suitable memory-hierarchy techniques such as cache optimization, appropriate memory technology selection, data locality, paging optimization, and hierarchical memory organization.
- Compute performance measures such as memory access time, latency, bandwidth, throughput, speedup, hit/miss rates, and resource or power utilization where applicable.
- Interpret the results by evaluating performance improvements, memory bottlenecks, technology trade-offs, and the suitability of the proposed memory hierarchy for each application.

QUESTIONS

1. A high-frequency financial trading system must process large volumes of market data with extremely low response time. Analyze the roles of CPU caches, DRAM, and persistent storage in handling frequently accessed trading data. Considering latency, data locality, capacity, and bandwidth, propose a memory hierarchy that minimizes data-access delays and supports high transaction throughput.
2. A medical imaging workstation processes large CT and MRI datasets while simultaneously running image-enhancement and visualization applications. Evaluate the interaction between cache memory, DRAM, and high-capacity storage during image processing. Propose an optimized memory hierarchy that balances access latency, memory capacity, bandwidth, and cost to improve overall processing performance.
3. A smart surveillance system receives continuous video streams from multiple high-resolution cameras and must detect objects in real time. Analyze the memory requirements for storing video frames, intermediate processing data, and frequently accessed detection parameters. Compare cache, DRAM, and non-volatile memory technologies and design a hierarchy that minimizes memory bottlenecks while maintaining high data throughput.
4. A scientific computing system performs large-scale simulations involving matrices and continuously accessed numerical datasets. Analyze how cache capacity, data locality, cache misses, DRAM bandwidth, and memory latency affect the execution of computational kernels. Recommend memory-hierarchy improvements that can reduce cache misses, increase effective memory bandwidth, and improve overall simulation performance.
5. A wearable health-monitoring device continuously collects sensor readings while operating under strict battery constraints. Analyze the trade-offs among SRAM, DRAM, and non-volatile memory in terms of latency, capacity, energy consumption, and data retention. Design a memory hierarchy that provides rapid access to frequently processed sensor data while minimizing power consumption and extending battery life.

Code of Conduct

*I **K.MAHESH BABU(192525122)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: K.MAHESH BABU(192525122)_____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Q. No	Understanding of Industry Problem (20)	Application of Engineering Concepts (25)	Solution Design (25)	Use of Tools (15)	Analysis (10)	Professionalism(5)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10		/ 100

Course Coordinator

Faculty Incharge

MEMORY HIERARCHY – APPLICATION-BASED QUESTIONS

1. High-Frequency Financial Trading System

Introduction

A high-frequency financial trading system processes a very large number of market-data messages and trading instructions within extremely short time intervals. Examples of frequently changing information include stock prices, bid and ask values, order-book entries, market indicators, and trading rules. Even a small memory delay can affect the time at which an order is generated and executed. Therefore, the system needs a memory hierarchy that provides extremely low latency for frequently accessed information while also providing sufficient capacity for large datasets.

Role of CPU Cache

CPU cache is the first major memory level used to reduce the delay between the processor and main memory. L1 cache is the smallest and fastest cache and should contain the most frequently used instructions and values. L2 cache provides a larger working area with slightly higher latency. L3 cache is usually larger and may be shared among processor cores. In a trading application, current prices, active order-book information, frequently executed trading rules, counters, and intermediate calculations can benefit from cache storage.

Role of DRAM

DRAM acts as the main working memory of the trading application. It can hold a much larger amount of data than cache, including current market feeds, recent price history, active trading models, order queues, and application buffers. DRAM has higher latency than cache, so data that is accessed repeatedly should be organized to maximize cache reuse. High-bandwidth DRAM is useful when many market-data streams must be processed simultaneously.

Role of Persistent Storage

Persistent storage such as NVMe SSDs is used for large datasets that do not need to be accessed for every transaction. Historical market data, transaction logs, audit information, configuration files, and archived records can be stored on SSDs. Persistent storage has much greater capacity than DRAM but much higher access latency, so it should not be placed in the critical path of time-sensitive trading operations whenever possible.

Data Locality and Optimization

Data locality is extremely important in financial trading. Temporal locality means that recently used data is likely to be used again, while spatial locality means that nearby memory locations are likely to be accessed together. Order-book structures should therefore be designed to keep related data close together. Cache-friendly data structures, prefetching, memory alignment, and minimizing unnecessary data copying can reduce cache misses. Keeping frequently used data in CPU cache allows the processor to perform more transactions without waiting for DRAM.

Proposed Memory Hierarchy

The recommended hierarchy is:
CPU Registers → L1 Cache → L2 Cache → L3 Cache → High-Bandwidth DRAM → NVMe SSD/Persistent Storage.

Registers are used for immediate calculations. L1, L2, and L3 caches keep progressively larger sets of frequently used trading data. DRAM stores the active application working set, while NVMe SSD provides large persistent capacity for historical and logged information.

Performance and Throughput

The proposed hierarchy minimizes data-access delays by ensuring that the hottest data is placed at the fastest level. Cache hits provide very low latency, whereas cache misses require access to lower memory levels. By improving cache hit rate, reducing unnecessary memory movement, and using high-bandwidth DRAM, the system can process more market events per second. This directly supports high transaction throughput and improves the responsiveness of the trading platform.

Conclusion

A high-frequency trading system should prioritize latency over capacity for its most frequently accessed data. CPU cache and registers should hold the critical trading information, DRAM should contain active market datasets, and NVMe storage should retain large historical and persistent information. Proper locality and cache optimization make the memory hierarchy capable of supporting extremely fast transaction processing.

2. Medical Imaging Workstation

Introduction

A medical imaging workstation processes large CT and MRI datasets while simultaneously performing image reconstruction, enhancement, segmentation, analysis, and visualization. Medical images can contain millions of pixels or three-dimensional voxels. During processing, the same image regions may be accessed repeatedly. Therefore, memory latency, capacity, and bandwidth all have a significant effect on the overall performance of the workstation.

Interaction with CPU Cache

CPU cache stores small but frequently reused data. Image-processing algorithms often use filter coefficients, loop variables, transformation parameters, and small image blocks repeatedly. Keeping these values in L1 and L2 cache reduces the number of accesses to DRAM. L3 cache provides additional space for larger shared working sets, especially when several CPU cores participate in image processing.

Role of DRAM

DRAM should contain the active CT or MRI examination, reconstructed images, segmentation masks, temporary processing buffers, and visualization data. Large-capacity DRAM is important because the complete active dataset can often be much larger than CPU cache. If the system does not have enough DRAM, it may frequently access storage, which can significantly reduce performance.

Role of High-Capacity Storage

High-capacity NVMe SSD storage should be used for original scans, previous examinations, completed results, image databases, and temporary datasets that are not currently being processed. SSDs provide persistent storage and much higher capacity than DRAM. However, the workstation should avoid repeatedly reading small pieces of data from storage during time-critical image-processing operations.

Bandwidth and Latency

Medical image processing requires both low latency and high bandwidth. Low latency helps when the processor needs individual values or small data structures quickly. High bandwidth is important when large image arrays must be transferred continuously. A balanced system therefore uses cache for low-latency access, DRAM for high-bandwidth active processing, and NVMe storage for capacity.

Optimization Techniques

Image data can be divided into smaller tiles or blocks. A tile can be loaded into cache, processed repeatedly, and then replaced by another tile. This technique improves spatial and temporal locality and reduces cache misses. Prefetching can also load the next required image block before the processor requests it. Parallel processing and efficient memory allocation can further improve throughput.

Proposed Memory Hierarchy

The recommended hierarchy is:
CPU Registers → L1 Cache → L2 Cache → L3 Cache → High-Capacity DRAM → NVMe SSD → Long-Term Archive.

Frequently reused coefficients and image blocks stay in cache. The active patient dataset and intermediate results stay in DRAM. Original images and completed studies are stored on persistent storage.

Performance and Cost Balance

Using only very fast memory would be extremely expensive because cache has limited capacity and high cost per bit. Using only large storage would be too slow for real-time image enhancement and visualization. The hierarchy provides a practical balance: expensive fast memory is reserved for frequently accessed data, while cheaper high-capacity memory and storage hold

larger datasets.

Conclusion

An optimized medical imaging workstation should use cache for frequently reused image-processing data, large DRAM for active CT and MRI datasets, and NVMe storage for persistent medical images. Tiling, prefetching, parallel processing, and locality-aware algorithms can reduce memory delays and improve overall image-processing performance.

3. Smart Surveillance System

Introduction

A smart surveillance system receives continuous video streams from multiple high-resolution cameras and performs real-time object detection. The system may identify people, vehicles, objects, or unusual events. Since frames arrive continuously, memory must be capable of handling high data rates without creating bottlenecks. The system must also retain selected video for later investigation.

Video Frame Storage Requirements

Each camera generates a sequence of image frames. High resolution and high frame rates increase the amount of memory required for buffering. The system may need to hold the current frame, previous frames, decoded frames, and frames waiting for processing. Multiple cameras multiply this requirement. Therefore, DRAM must have sufficient capacity and bandwidth to support simultaneous streams.

Role of Cache

Cache is used for frequently accessed detection parameters, loop instructions, neural-network control values, small image blocks, and intermediate calculations. When the same parameters or data are reused repeatedly, cache access prevents the CPU from repeatedly accessing DRAM. A high cache hit rate improves the speed of object-detection operations.

Role of DRAM

DRAM stores current video frames, frame buffers, decoded images, feature maps, and intermediate results generated by object-detection algorithms. Double buffering can be used so that one buffer receives a new frame while another buffer is processed. This allows capture and processing to continue simultaneously.

Role of Non-Volatile Memory

NVMe SSD or another non-volatile storage device should store recorded video, event clips, detection logs, timestamps, and historical footage. Unlike DRAM, non-volatile storage retains data after power is removed. Because storage is slower, only required frames or event clips should be written during real-time operation, rather than repeatedly moving all data through slow storage paths.

Memory Technology Comparison

Cache provides the lowest latency but has very small capacity and high cost. DRAM offers a good combination of capacity, bandwidth, and latency and is suitable for active video processing. Non-volatile storage provides very high capacity and permanent retention but has higher latency. Combining these technologies creates a balanced system.

Techniques to Reduce Bottlenecks

Direct Memory Access (DMA) can transfer camera data into memory with reduced CPU involvement. Double buffering prevents the processor from waiting for frame capture. Frame batching can improve processing efficiency when the AI model supports it. Compression can reduce the amount of data written to persistent storage. Efficient memory allocation also prevents unnecessary copying of frames.

Proposed Memory Hierarchy

The recommended hierarchy is:
CPU Registers → L1/L2/L3 Cache → DRAM → NVMe SSD/Persistent Storage.

Detection parameters and frequently reused values stay in cache. Current frames and AI intermediate results stay in DRAM. Recorded footage and logs are transferred to SSD storage.

Real-Time Performance

The main objective is to prevent the arrival of new video frames from exceeding the system's processing capacity. If memory bandwidth is insufficient, frames may be delayed or dropped. A high-bandwidth DRAM subsystem, efficient buffering, and cache-friendly AI processing help maintain real-time detection and high data throughput.

Conclusion

The surveillance system should keep frequently accessed detection information in cache, active video frames and intermediate AI data in DRAM, and historical video in non-volatile storage. DMA, double buffering, compression, and efficient frame management can minimize memory bottlenecks and maintain real-time performance.

4. Scientific Computing System

Introduction

Scientific computing systems perform simulations involving large matrices, numerical datasets, equations, and computational models. Examples include weather simulation, fluid dynamics, molecular modeling, structural analysis, and engineering calculations. These applications repeatedly access large amounts of data, making memory hierarchy an important factor in execution time.

Cache Capacity and Data Locality

Many scientific algorithms access nearby elements of matrices repeatedly. If the working data fits into cache, the processor can reuse it without repeatedly accessing DRAM. Temporal locality occurs when the same data is reused, while spatial locality occurs when nearby elements are accessed. Efficient algorithms should be designed to maximize both forms of locality.

Effect of Cache Misses

A cache miss occurs when the processor requests data that is not currently stored in the required cache level. The system then has to retrieve the data from a lower memory level. Frequent cache misses increase memory latency and reduce CPU utilization. In matrix computations, poor loop ordering can cause unnecessary cache misses and significantly increase execution time.

DRAM Bandwidth

Large scientific datasets often cannot fit entirely into cache. Therefore, DRAM bandwidth becomes important. If the processor can perform calculations faster than DRAM can supply data, the application becomes memory-bound. High-bandwidth DRAM and efficient data access patterns help supply data more continuously to the processor.

Cache Blocking or Tiling

Cache blocking divides a large matrix into smaller blocks that fit into cache. Instead of processing an entire large matrix at once, the algorithm processes one block repeatedly before moving to another. This improves data reuse and reduces the number of times data must be loaded from DRAM. Blocking is especially useful for matrix multiplication and other numerical kernels.

Other Improvements

Other improvements include sequential memory access, memory alignment, prefetching, reducing unnecessary copies, using compact data structures, and parallelizing computations across processor cores. These techniques reduce memory stalls and increase effective memory bandwidth.

Proposed Memory Hierarchy

The recommended hierarchy is:
CPU Registers → L1 Cache → L2 Cache → L3 Cache → High-Bandwidth DRAM → Persistent Storage.

Registers hold immediate operands, cache holds frequently reused matrix blocks, DRAM holds the main numerical dataset, and persistent storage holds input files, checkpoints, and simulation results.

Effect on Computational Kernels

Computational kernels are the most frequently executed parts of a scientific program. If the required data is already in cache, the processor can execute arithmetic operations quickly. If every operation requires a DRAM access, execution slows down because the processor spends more time waiting for data. Therefore, improving locality can provide a major performance improvement without changing the mathematical result.

Conclusion

Scientific applications benefit greatly from a memory hierarchy that emphasizes locality and data reuse. Cache blocking, efficient loop organization, prefetching, high DRAM bandwidth, and reduced data movement can lower cache misses and improve the performance of large-scale simulations.

5. Wearable Health-Monitoring Device

Introduction

A wearable health-monitoring device continuously collects sensor measurements while operating with strict battery limitations. Sensors may monitor heart rate, body temperature, movement, activity, or other signals. The device must process current readings quickly, store important information, and consume as little energy as possible. Therefore, memory design must consider latency, capacity, power consumption, and data retention together.

Role of SRAM and Cache

SRAM is useful for frequently accessed sensor readings, program variables, buffers, and control information. It provides fast access and does not require the periodic refresh operation used by DRAM. However, SRAM occupies more chip area per bit and is expensive for large capacities. Therefore, only the most frequently used working data should be kept in SRAM or cache.

Role of DRAM

DRAM can provide larger temporary storage when the device needs to process more data. However, refresh operations and active memory accesses consume energy. For a small wearable device, the system should avoid keeping unnecessary DRAM active. If the workload is small enough, low-power SRAM may be preferable for frequently used data.

Role of Non-Volatile Memory

Flash or other non-volatile memory is useful for long-term sensor history, configuration information, event records, and data that must survive power loss. Non-volatile memory provides large persistent capacity but is slower than SRAM. It is therefore better suited for less frequently accessed data rather than real-time processing.

Latency, Capacity, and Energy Trade-Off

Fast memory generally has higher cost and limited capacity, while large persistent memory is slower. The wearable must therefore keep only the most important and frequently accessed data in fast memory. Sensor data can be processed in a small RAM buffer and then written to flash in batches. This reduces repeated memory and storage operations and saves energy.

Battery-Life Optimization

The device can use sleep modes when there is no active processing. Sampling rates can be adjusted according to the application, and unnecessary sensor readings can be filtered before storage. Data compression can reduce storage requirements. Batch writes to flash are preferable to frequent small writes because they reduce overhead and allow the system to spend more time in low-power states.

Proposed Memory Hierarchy

The recommended hierarchy is:
CPU Registers → Small SRAM/Cache → Low-Power Working Memory → Non-Volatile Flash Storage.

Registers are used for immediate sensor calculations. SRAM/cache holds active readings and program variables. Low-power working memory provides additional temporary space when required. Flash stores historical data and information that must be retained after power loss.

Data Retention and Reliability

Health-monitoring data may need to be retained for later analysis. Important events can be stored permanently in flash, while temporary sensor readings can be discarded after processing if they are not needed. Wear-leveling and careful write management can help extend flash memory life. Error detection and periodic data validation can also improve reliability.

Conclusion

The wearable device should use a small amount of very fast SRAM/cache for frequently processed sensor data and low-power non-volatile flash for historical information. Buffering, batch writes, sleep modes, compression, and efficient sampling reduce energy consumption while maintaining rapid access and reliable data retention. This hierarchy supports both real-time monitoring and long battery life.

Summary of Recommended Memory Hierarchies

Application	Main Working Memory	Recommended Hierarchy
Financial Trading	Cache + High-bandwidth DRAM	Registers → Cache → DRAM → NVMe SSD
Medical Imaging	High-capacity DRAM	Registers → Cache → DRAM → NVMe SSD → Archive
Smart Surveillance	DRAM for video buffers	Registers → Cache → DRAM → NVMe SSD
Scientific Computing	High-bandwidth DRAM	Registers → Cache → DRAM → Persistent Storage
Wearable Device	Low-power SRAM	Registers → SRAM/Cache → Working Memory → Flash

Overall Conclusion

The five applications demonstrate that there is no single memory technology that is ideal for every requirement. Cache and registers provide extremely low latency but have limited capacity. DRAM provides a larger and faster working area for active datasets, while non-volatile storage provides high capacity and permanent data retention. The most effective memory hierarchy places frequently accessed data in fast memory and less frequently accessed data in larger, slower memory. Techniques such as data locality, cache blocking, prefetching, buffering, DMA, compression, batching, and power-saving modes can further improve performance. Therefore, memory hierarchy design should always consider the application's latency, capacity, bandwidth, cost, reliability, and energy requirements.